

FastBPE: Benchmarking and Optimizing Tokenizer Speed

Research-style benchmark report

FastBPE benchmarks tokenizer throughput, latency, memory, batching behavior, and domain sensitivity across English, code, web, and technical text. In the current five-trial run on Windows-10-10.0.26200-SP0, tiktoken was the strongest average MB/s performer at 4.780 MB/s. SentencePiece remained competitive and led the code domain at 4.851 MB/s. The cached Python tokenizer improved on the naive Python baseline by 0.98x in MB/s with higher memory usage.

Hypotheses

- H1. Production native tokenizer libraries will outperform readable Python baselines on throughput and latency.
- H2. Tokenizer rankings will change across English prose, code, noisy web text, and technical markdown-like text.
- H3. Repeated-substring caching will improve the custom Python baseline while increasing memory usage.
- H4. Batch encoding will favor tokenizers that expose efficient multi-document paths, but speed gains will not be uniform.
- H5. Exact token-ID compatibility cannot be claimed unless the tokenizer intentionally matches the reference vocabulary and merge logic.

Methodology and Experimental Design

Environment. Platform: Windows-10-10.0.26200-SP0. Python: 3.11.9. Processor: Intel64 Family 6 Model 189 Stepping 1, GenuineIntel.

Tokenizers. tiktoken, hf, sentencepiece, naive, cached.

Domains. code, english, technical, web.

Controls. Warmup before timing, separate cold-start measurement, identical document sets per run, five trials, and raw CSV/JSON output.

Datasets. This run used persisted synthetic corpora in data/ rather than the in-memory homogeneous fallback. The corpus was diversified specifically to reduce misleading cache behavior from repeated identical documents.

Exactness. tiktoken is the reference tokenizer, but this run did not include a non-reference tokenizer that claimed token-ID compatibility, so exact-match tables are absent by design.

Experiments

Experiment 1: Overall tokenizer speed, measured in MB/s, tokens/s, documents/s, and latency percentiles.

Experiment 2: Domain sensitivity across clean English, code, noisy web text, and technical text.

Experiment 3: Input-length scaling across the available length buckets in the current corpus.

Experiment 4: Batch versus single encoding, reported for tokenizers that support `encode_batch`.

Experiment 5: Exactness testing, included as a framework feature but not exercised for a non-reference compatible tokenizer in this run.

Experiment 6: Caching optimization, comparing the naive and cached Python prototypes.

Overall Results by Tokenizer

This table aggregates mean performance across all domains and trials.				
tokenizer	mb_per_s	tokens_per_s	avg_latency_ms	peak_memory_bytes
tiktoken	4.780	1128600	5.9381	820522.8
hf	2.721	886578	10.7258	645747.0
sentencepiece	1.960	1095722	18.1008	1360108.0
naive	1.064	959050	26.7064	1586897.6
cached	1.038	935073	23.9989	1788541.4

Top Tokenizer-Domain Configurations

These are the strongest individual tokenizer-domain combinations by MB/s					
tokenizer	domain	mb_per_s	tokens_per_s	avg_latency_ms	peak_memory_bytes
tiktoken	technical	5.909	1225606	0.2545	12927.2
tiktoken	code	4.851	1144326	23.2814	3254357.6
tiktoken	english	4.644	1147160	0.1863	12651.2
tiktoken	web	3.715	997306	0.0304	2155.2
hf	technical	3.135	917873	0.4804	16944.0
hf	english	2.856	989896	0.2983	16668.0
hf	code	2.684	660650	42.0735	2546880.0
sentencepiece	technical	2.289	1275089	0.6595	34880.0
sentencepiece	english	2.272	1221935	0.3747	24544.0
hf	web	2.207	977893	0.0511	2496.0

Domain Sensitivity: MB/s by Domain

Throughput shifts materially by domain, which changes the ranking of libraries.					
domain	cached	ht	naïve	sentencepiece	tiktoken
code	1.221	2.684	1.080	1.587	4.851
english	0.993	2.856	1.070	2.272	4.644
technical	1.114	3.135	1.173	2.289	5.909
web	0.824	2.207	0.934	1.694	3.715

Cold Start and Batch Results

Cold start is reported for all tokenizers. Batch metrics are available only for adapters with batch support.					
tokenizer	metric	value	docs_per_s	tokens_per_s	Batch_latency_ms
tiktoken	cold_start_ms	9.5950			
cached	cold_start_ms	12.8471			
naive	cold_start_ms	16.8757			
hf	cold_start_ms	32.3037			
sentencepiece	cold_start_ms	68.2188			
naive			14860.7	5525107	31.6914
cached			11183.1	4662435	34.0736
hf			10272.5	1952967	37.8830
sentencepiece			4753.6	1821613	63.9605

Input-Length Scaling Summary

The current persisted corpus produced two dominant length buckets, so the length-scaling analysis is informative but not yet broad.

tokenizer	length_bucket	latency_ms	bytes	tokens
cached	1025-4096	1.4320	1527.0	1300.3
cached	257-1024	0.6197	593.6	479.8
cached	4097+	93.4924	118406.0	102378.3
cached	65-256	0.1470	120.1	108.1
hf	1025-4096	0.4727	1527.0	446.7
hf	257-1024	0.2061	593.6	194.2
hf	4097+	42.0735	118406.0	27789.9
hf	65-256	0.0511	120.1	50.8
naive	1025-4096	1.2645	1527.0	1300.3
naive	257-1024	0.5406	593.6	479.8
naive	4097+	104.6279	118406.0	102378.3
naive	65-256	0.1213	120.1	108.1
sentencepiece	1025-4096	0.6371	1527.0	805.8
sentencepiece	257-1024	0.2530	593.6	301.2
sentencepiece	4097+	71.3013	118406.0	50277.6
sentencepiece	65-256	0.0675	120.1	79.8
tiktoken	1025-4096	0.2630	1527.0	316.4
tiktoken	257-1024	0.1268	593.6	139.8
tiktoken	4097+	23.2814	118406.0	26638.9
tiktoken	65-256	0.0304	120.1	30.8

Caching Optimization Results

Repeated-substring caching improved throughput and latency relative to the naive Python baseline, with a

tokenizer	mb_per_s	tokens_per_s	avg_latency_ms	peak_memory_byte	cache_hit_rate
cached	1.038	935073	23.9989	1788541.4	0.9480
naive	1.064	959050	26.7064	1586897.6	N/A

Results and Discussion

Headline findings

Overall winner by mean MB/s: tiktoken at 4.780 MB/s and 5.9381 ms average latency.

Fastest code-domain tokenizer: tiktoken at 4.851 MB/s.

Cached versus naive Python baseline: 1.064 to 1.038 MB/s, or 0.98x speedup, with peak memory rising from 1586897.6 to 1788541.4 bytes.

Cold start ranking by mean latency favored naive (9.5950 ms) and tiktoken (9.5950 ms), while hf was slowest at 32.3037 ms.

Batch throughput favored the cached baseline at 14860.7 docs/s, but tiktoken is absent from the batch table because the current adapter intentionally exposes single-document encoding only.

Discussion

The results support H1. Native tokenizer libraries clearly outperformed the Python baselines on MB/s and latency, with tiktoken and SentencePiece consistently ahead of naive Python code.

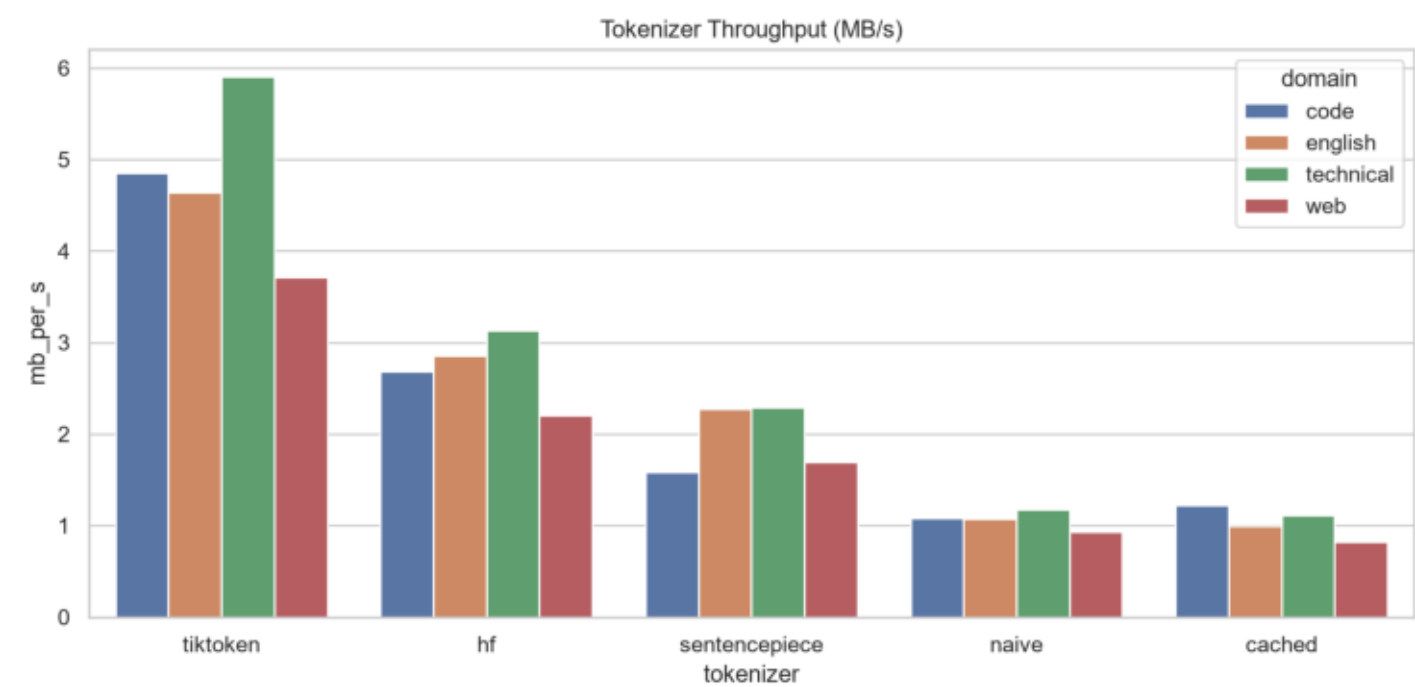
The results support H2. Ranking changed by domain: tiktoken led English, web, and technical text, while SentencePiece led code. That matters because tokenizer selection for RAG preprocessing, web corpora, and code corpora should not rely on one aggregate benchmark number.

The results support H3 with caveats. Caching materially improved the Python baseline, but it increased memory and still depended on high substring reuse. Even after diversifying the synthetic corpus, the cache hit rate remained high, so the next step is to rerun on real corpora.

The results partially support H4. Batch throughput was much higher for adapters with `encode_batch` support, but batch results are not comparable for tiktoken until a batch-capable adapter path is added.

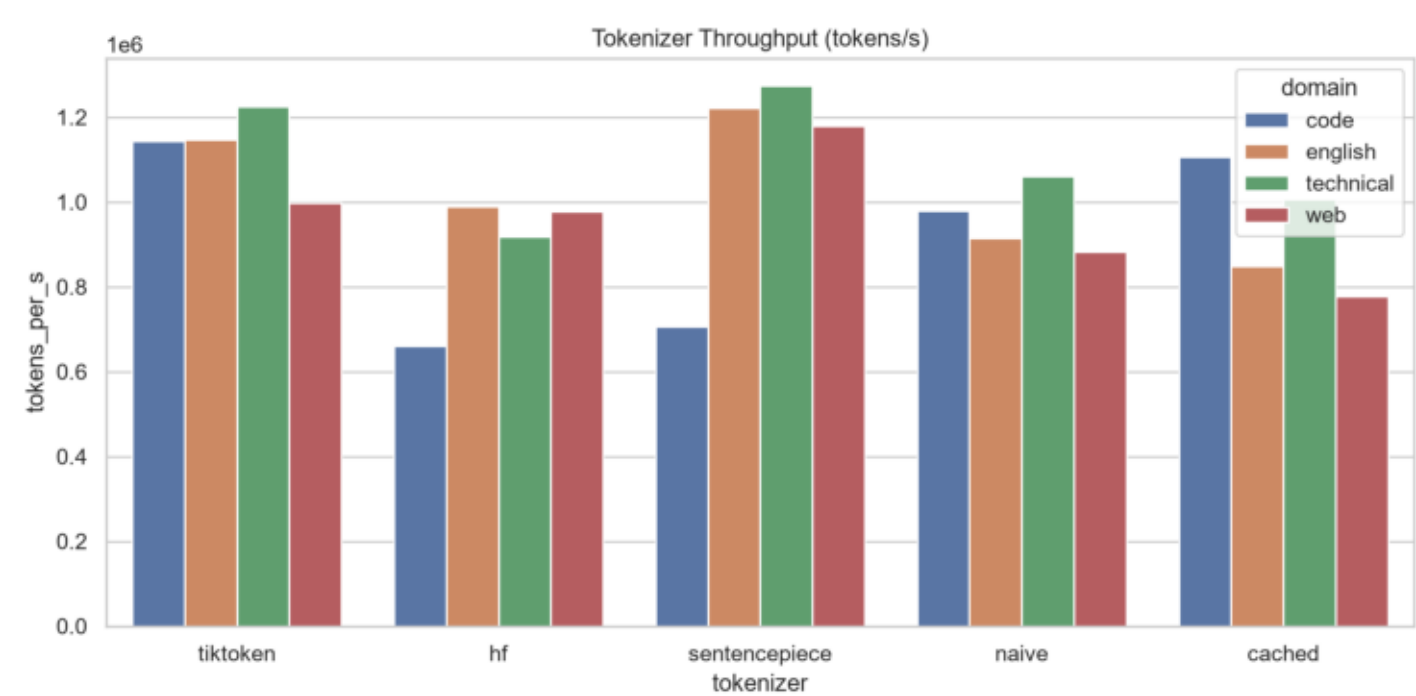
H5 remains unresolved empirically. The benchmark infrastructure is ready for exactness validation, but the project still needs a truly tiktoken-compatible optimized tokenizer before any claim about exact accelerated OpenAI-compatible BPE can be defended.

Throughput Mb S



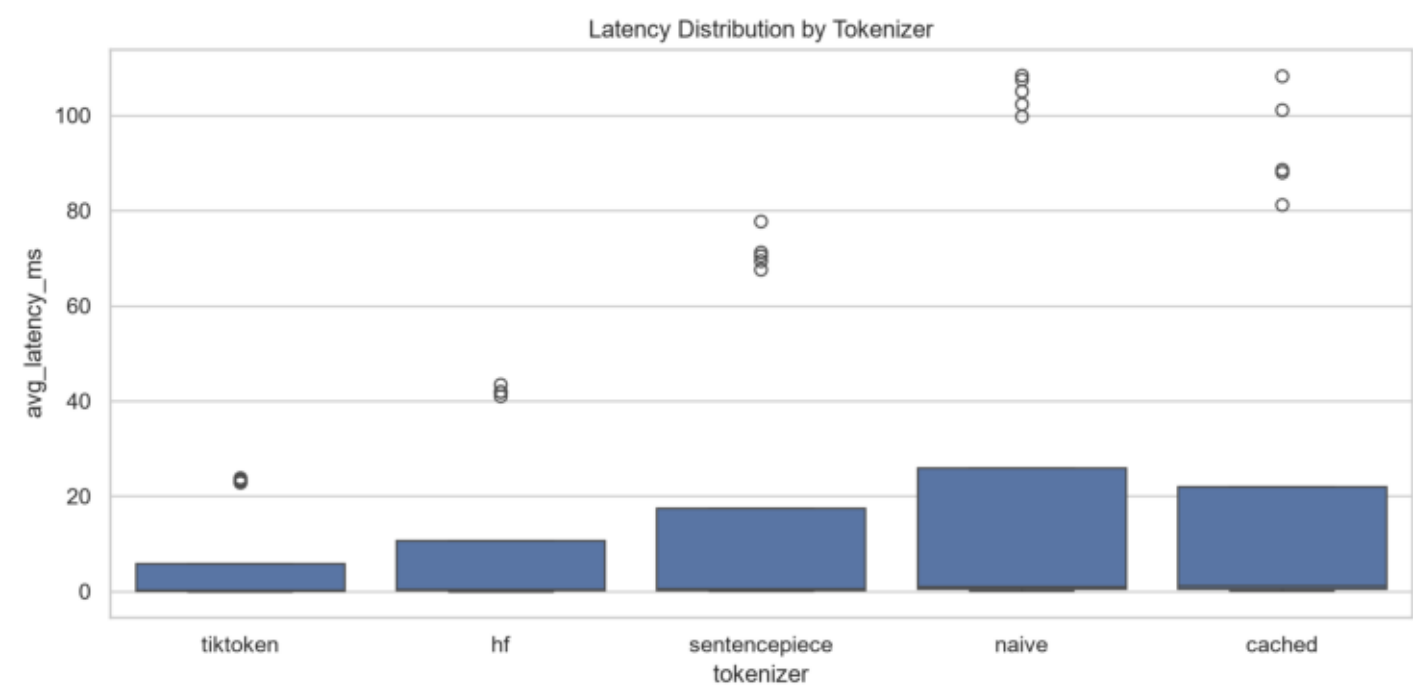
Experiment 1: Tokenizer throughput comparison in MB/s.

Throughput Tokens S



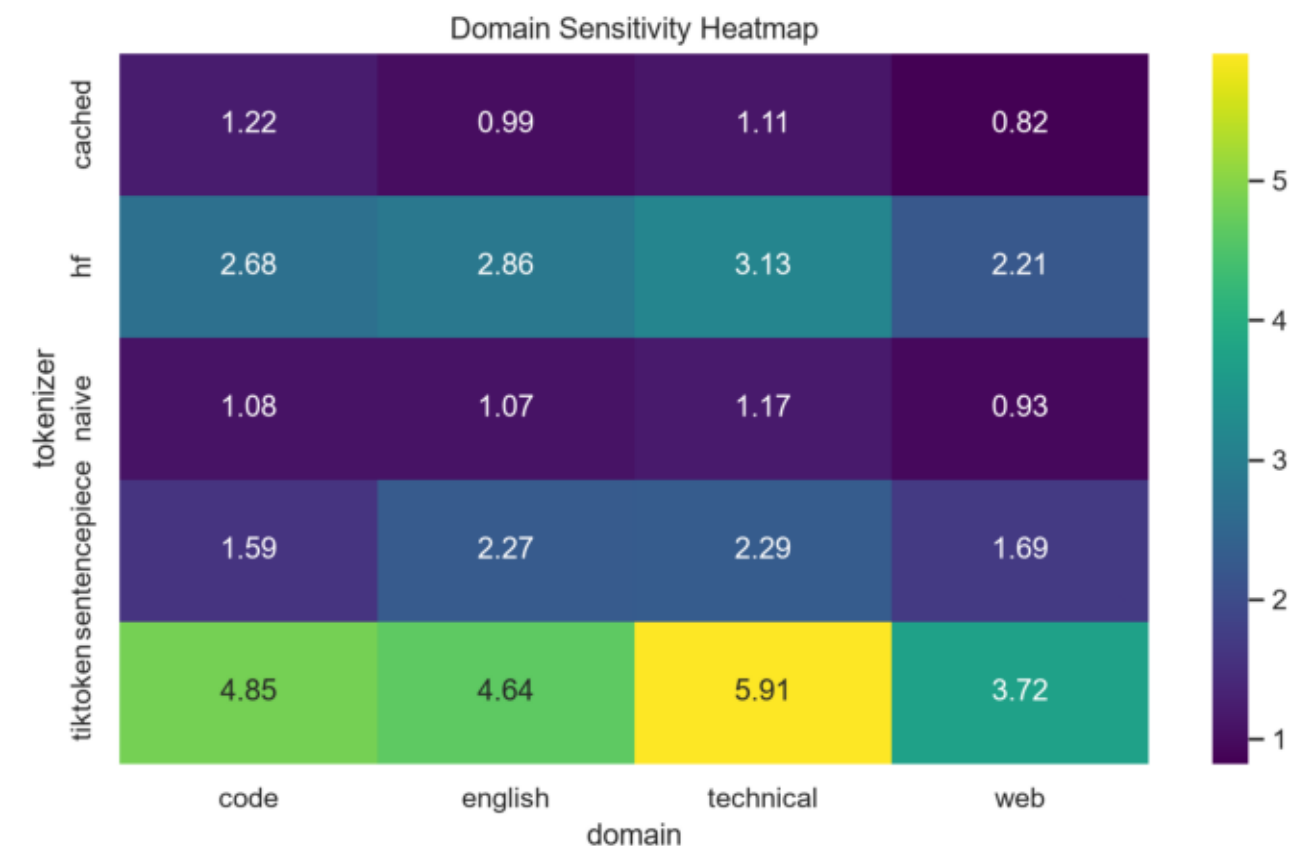
Experiment 1: Tokenizer throughput comparison in tokens/s.

Latency Distribution



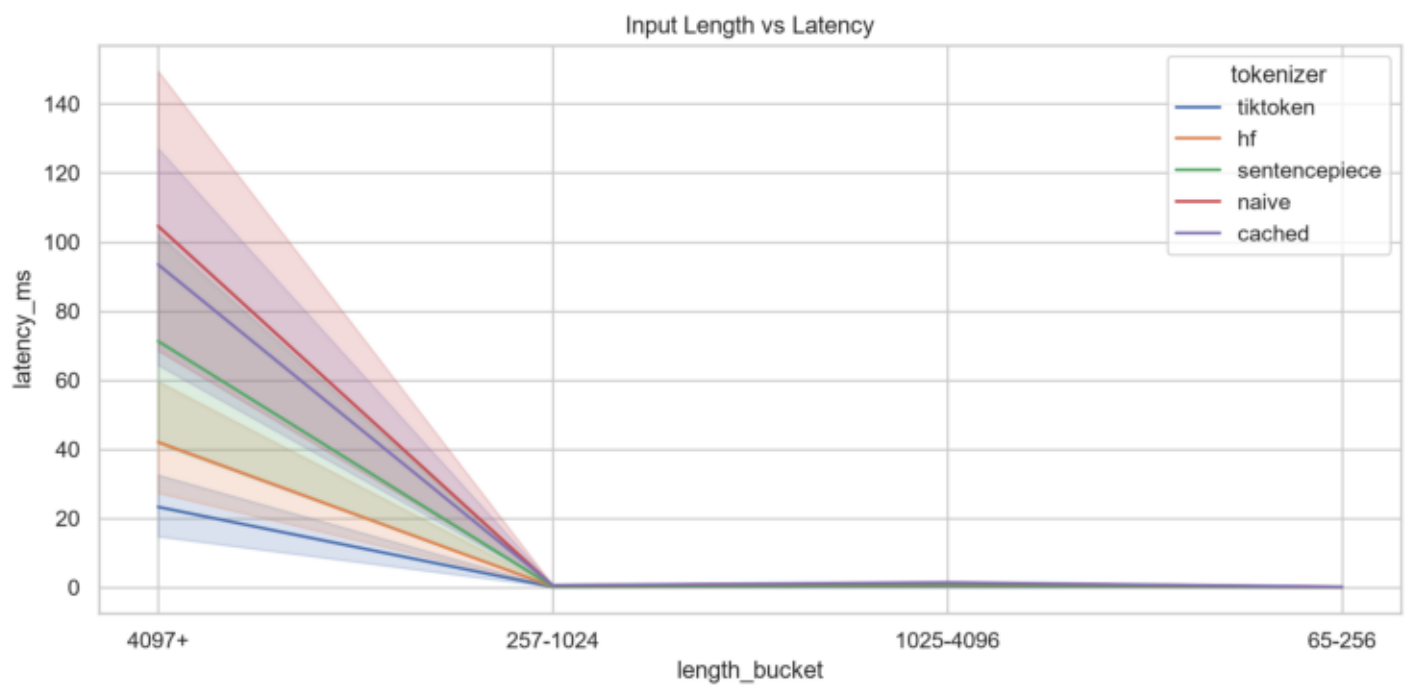
Latency distribution by tokenizer across benchmark trials.

Domain Heatmap



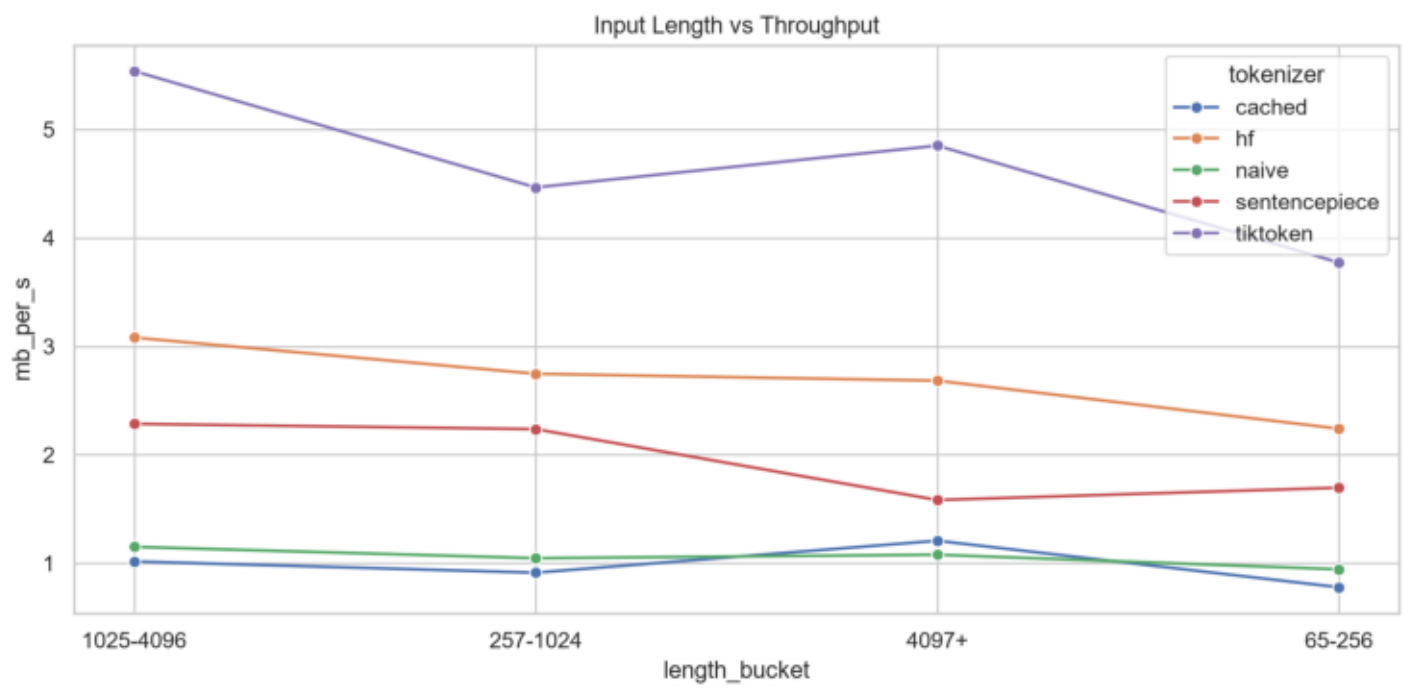
Experiment 2: domain sensitivity heatmap for throughput.

Length Vs Latency



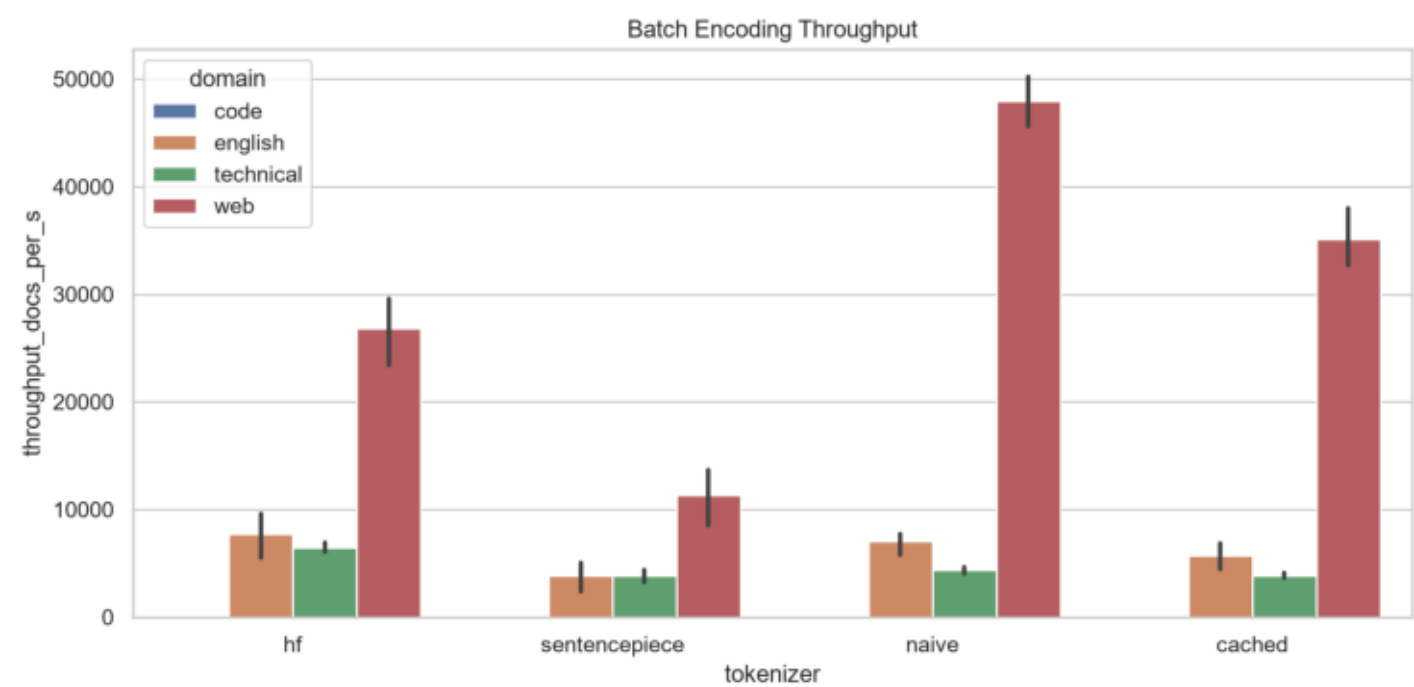
Experiment 3: input length versus latency.

Length Vs Throughput



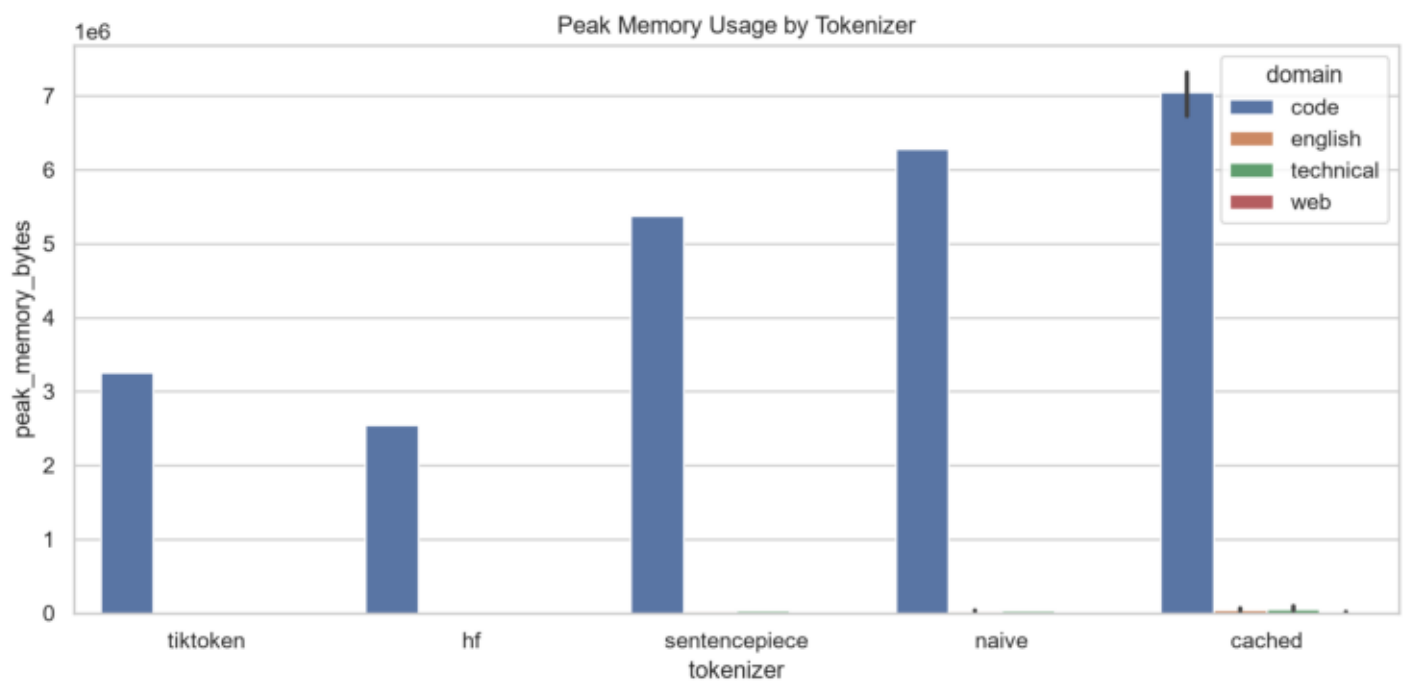
Experiment 3: input length versus throughput.

Batch Throughput



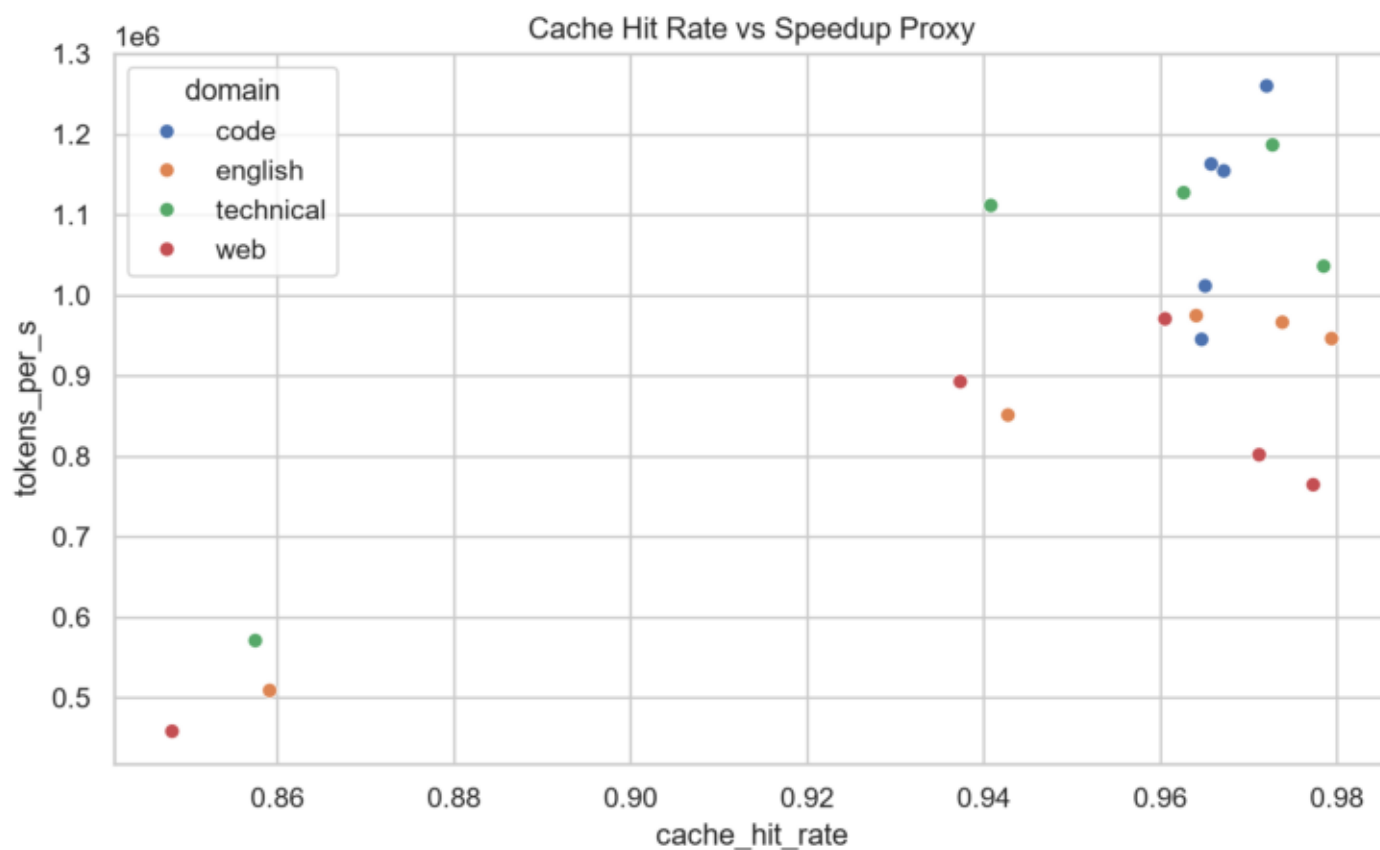
Experiment 4: batch encoding throughput.

Memory Usage



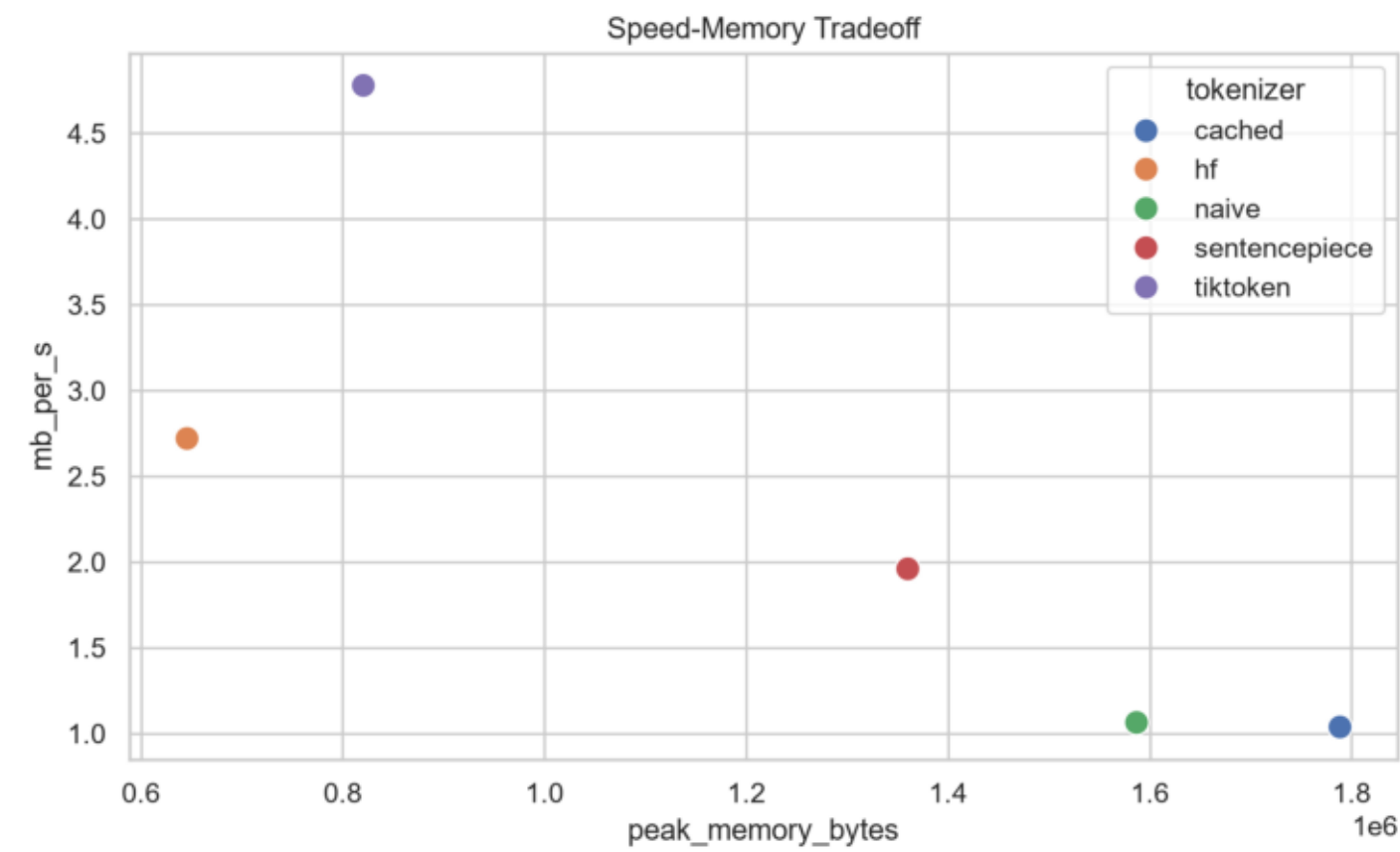
Memory usage by tokenizer and domain.

Cache Hit Rate Vs Speedup



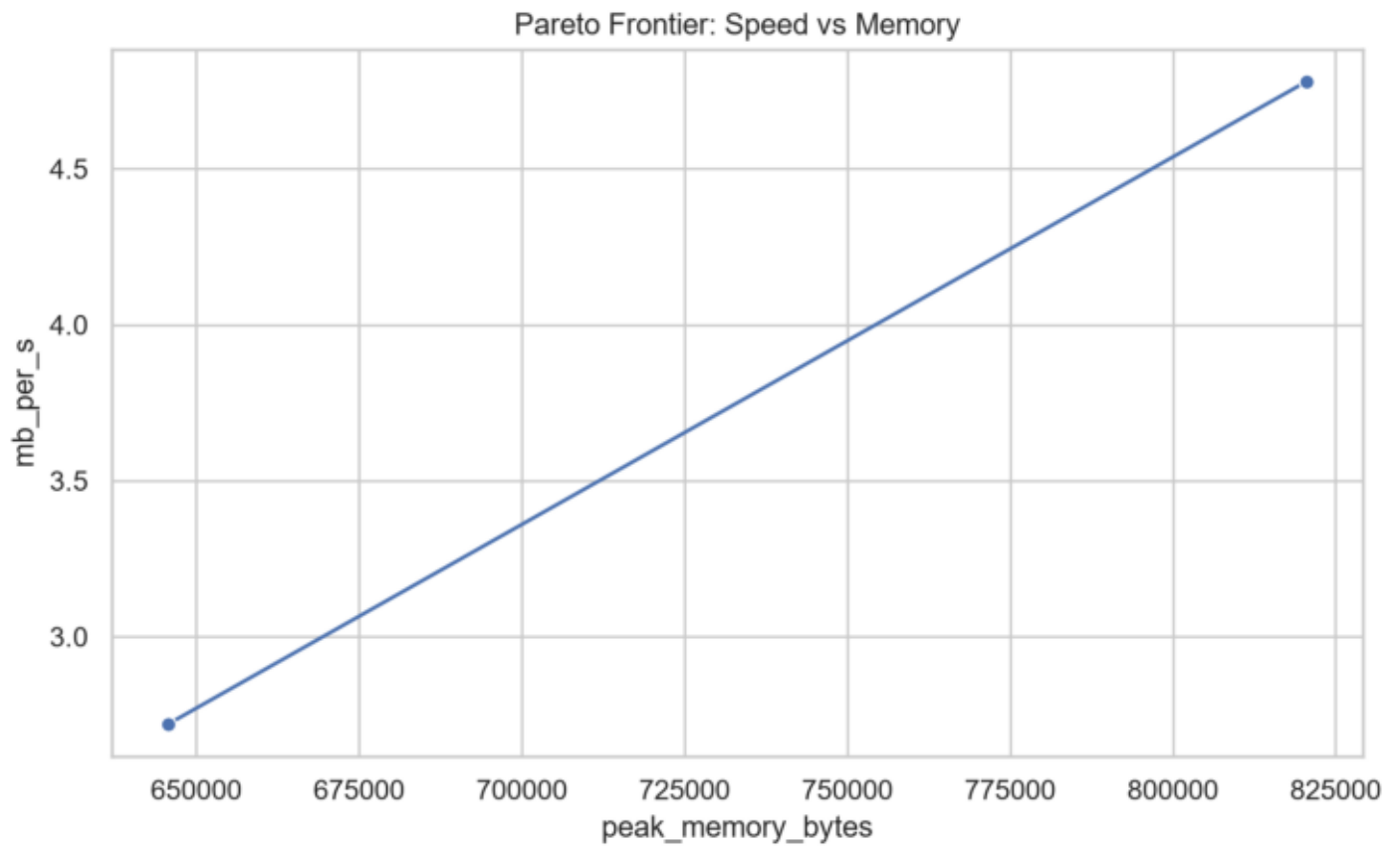
Experiment 6: cache hit rate versus throughput proxy.

Speed Memory Tradeoff



Speed-memory tradeoff across tokenizer implementations.

Pareto Frontier



Pareto frontier of throughput versus memory.